

EJERCICIO 1

Simplemente consiste en la creación de un repositorio y subirlo a GitHub

Comandos utilizados:

- git init: Para inicializar el repositorio
- git add : Para añadir al espacio intermedio temporal el archivo java
- git commit -m “mensaje”: Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git remote add origin: Permite indicar cuál es el origen del repositorio remoto para poder subirlo

No me dio errores en este ejercicio

EJERCICIO 2

Consiste en crear un repositorio en GitHub y clonarlo

Comandos utilizados:

- git clone <url> : Es para clonar el repositorio indicado con su respectiva url
- git status: Comprueba el estado actual
- git add . : Para añadir al espacio intermedio los archivos
- git commit -m “mensaje”: Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git push: Empuja la rama actual a su rama de seguimiento

Tuve problemas para clonar el repositorio pero pude solucionarlo creando un nuevo repositorio en GitHub y borrando el primero

EJERCICIO 3

Se trata de la edición del archivo en el ejercicio 2 y realizar un seguimiento

Comandos utilizados:

- git status: Comprueba el estado actual; para saber desde el inicio cómo se encontraba
- git diff: Sirve para mostrar cambios entre commits y distinguir las diferencias entre ellos

- git add : Para añadir al espacio intermedio temporal el archivo java
- git commit -m “mensaje”: Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git push: Empuja la rama actual a su rama de seguimiento

No me dio errores en este ejercicio

EJERCICIO 4

Consiste en crear un archivo java y crear una nueva rama que subir

Comandos utilizados:

- git checkout -b <name> : Crea la nueva rama requerida y cambia inmediatamente a ella
- git add : Para añadir al espacio intermedio temporal el archivo java
- git commit -m “mensaje”: Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git push -u origin <rama> : Empuja la rama hacia el control remoto
- git remote add origin: Permite indicar cuál es el origen del repositorio remoto para poder subirlo

Me costó subir la rama al repositorio ya que había un error de dirección con el directorio, para solucionarlo tuve que usar el comando git remote remove origin para reindicarle manualmente dónde debe ir

EJERCICIO 5

El ejercicio trata de fusionar una rama en GitHub y gestionarlo con ayuda desde Git

Comandos utilizados:

- git checkout main: Debía cambiar a la rama principal ya que me encontraba en la otra rama por el ejercicio anterior
- git pull: Sirve para bajar los cambios generados en GitHub
- git branch -d <name> : Borra una rama

No me dio errores en este ejercicio

EJERCICIO 6

El ejercicio es crear un conflicto y resolverlo

Comandos utilizados:

- git add : Para añadir al espacio intermedio temporal el archivo java
- git commit -m "mensaje": Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git pull: Sirve para bajar los cambios generados en GitHub
- git push: Empuja la rama actual a su rama de seguimiento

Tuve bastantes problemas al usar git push ya que no reconocía la rama o algo así, desconozco por qué

EJERCICIO 7

Consiste en utilizar los comandos fetch y pull

Comandos utilizados:

- git fetch: Lo que hace es descargae los últimos commits, y archivos de un repositorio remoto a tu repositorio local, actualizando las ramas pero sin modificar la rama de trabajo actual
- git log origin/main: Muestra el historial de commits de la rama main en el repositorio remoto llamado origin, es decir, los cambios que se han hecho y confirmado
- git pull: Sirve para bajar los cambios generados en GitHub

No me dio errores en este ejercicio

EJERCICIO 8

El ejercicio trata de crear un par de archivos chiquitos para posteriormente borrarlos y que quede notificado en GitHub

Comandos utilizados:

- git add : Para añadir al espacio intermedio temporal el archivo java
- git rm : Borra el archivo que le indiques
- git commit -m "mensaje": Para realizar una confirmación y escribiendo un mensaje al mismo tiempo

- git push: Empuja la rama actual a su rama de seguimiento

El comando git push no funcionaba correctamente y no tengo claro si git pudo identificar las acciones realizadas. Pero no encontré los archivos, por lo que fueron correctamente eliminados

EJERCICIO 9

El ejercicio es crear un nuevo archivo para ignorar archivos temporales y compilados con .gitignore

Comandos utilizados:

- touch .gitignore: Crea un nuevo archivo llamado .gitignore en el directorio actual. Este archivo se usa para indicarle a Git qué archivos debe ignorar.
- echo "*.class" >> .gitignore: Escribe el patrón *.class dentro del archivo .gitignore. Esto le dice a Git que ignore cualquier archivo que termine en la extensión .class
- echo "*.log" >> .gitignore: Escribe el patrón *.log dentro del archivo .gitignore. Esto le indica a Git que ignore cualquier archivo que termine en la extensión .log
- touch MiClase.class: Crea un nuevo archivo vacío llamado MiClase.class. Git lo ignorará gracias a la regla *.class que se añadió anteriormente.
- touch error.log: Crea un nuevo archivo vacío llamado error.log. Git lo ignorará gracias a la regla *.log que se añadió anteriormente.
- git status: Comprueba el estado actual; para saber desde el inicio cómo se encontraba
- git add : Para añadir al espacio intermedio temporal el archivo java
- git commit -m "mensaje": Para realizar una confirmación y escribiendo un mensaje al mismo tiempo
- git push: Empuja la rama actual a su rama de seguimiento

No me dio errores en este ejercicio

EJERCICIO 10

El ejercicio trata de usar git reset para deshacer un cambio

Comandos utilizados:

- `git commit -am "Cambio erróneo"`: Sirve para realizar una confirmación de una edición equivocada
- `git reset HEAD 1`: Revierte el cambio que se acaba de hacer
- `git commit -am "Vuelvo a modificar"`: Otra confirmación para volver a hacer el cambio anterior
- `git push`: Empuja la rama actual a su rama de seguimiento

No me dio errores en este ejercicio